

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Department of Computer Science and Engineering
Artificial Intelligence

**GUIDING MOE REASONING VIA ROUTING
INTERVENTION**

Master Thesis in
Big Data Analytics and Text Mining

Supervisor

Prof. Gianluca Moro

Candidate

Leonardo Tassinari

Co-supervisors

Dr. Giacomo Frisoni

Dr. Lorenzo Molfetta

Second Session
Academic Year 2025 – 2026

KEYWORDS

Keyword 1

Keyword 2

Keyword 3

Keyword 4

Keyword 5

Abstract

Introduction

Contents

1	Theoretical Framework	1
1.1	Reasoning as a measurable process	1
1.2	Sparse mixture-of-experts computation	2
1.2.1	Experts, routers, and conditional computation	2
1.2.2	Interpreting expert identity	4
1.3	A vocabulary for reasoning episodes	4
1.3.1	Adapting episode theory to model-generated text	4
1.3.2	Automatic episode annotation	5
1.4	Prospective representation probes	6
1.5	Routing measurements and statistical analysis	7
1.5.1	Concentration, selection, and temporal dynamics	7
1.5.2	Binary correctness and repeated-attempt accuracy	9
1.5.3	Dependence, uncertainty, and confounding	10
1.6	From observations to routing intervention	11
2	Related Work	13
2.1	Reasoning traces, evaluation, and functional annotation	13
2.2	Prompt optimisation for episode annotation	14
2.3	Probing representations of reasoning	15
2.4	Expert specialisation and direct routing control	17
2.5	Activation steering	19
2.6	Statistical evidence for metric selection	20
2.7	Position of the experimental pipeline	21
3	Method	25
3.1	Optimising the LLM-as-judge prompt	25

3.2	Identifying relevant layers with prospective probes	28
3.3	Relating routing to reasoning and correctness	29
3.3.1	Generation, annotation, and replay	29
3.3.2	Measurements and analysis views	30
3.3.3	Statistical unit and outcome association	31
3.4	Guiding experiments	32
3.4.1	Calibration and held-out evaluation	32
3.4.2	Expert-identity guiding	33
3.4.3	Margin guiding	33
3.4.4	Matched generation and evaluation	34
4	Experimental Setup	37
4.1	Computing infrastructure	37
4.2	Software and model configurations	39
4.2.1	The subject model and the annotation models	39
4.2.2	Quantisation and generation–replay provenance	40
4.3	Datasets and units of observation	40
4.3.1	Annotated reasoning traces	40
4.3.2	Mathematical reasoning benchmarks	41
4.4	Episode-annotation protocol	42
4.5	Prospective hidden-state probes	44
4.6	Benchmark generation and routing extraction	45
4.6.1	Generation settings	45
4.6.2	Replay measurements and layer selection	45
4.6.3	Sentence annotation	46
4.7	Evaluation and statistical protocol	47
4.8	Artifacts, reproducibility, and experimental scope	48
5	Results	51
6	Conclusions	53
	Bibliography	55

List of Figures

3.1	Information flow through the method. Annotation and probing use the reference corpus independently. Their outputs meet in the correlation experiment; guiding policies are fitted using calibration problems only.	26
-----	--	----

List of Tables

1.1	Operational episode categories used by the project, based on the adapted framework of Li et al. [9]. The descriptions concern a unit’s function, rather than its mathematical validity.	5
2.1	Role of each stage in the routing-intervention pipeline. These are research functions, rather than claims of completed causal validation.	22
3.1	Probe-derived router locations used by the method. Indices are zero-based decoder indices and are specific to each model’s probe run.	29
4.1	Resources of the principal experimental node. GPU specifications follow the cluster guide; CPU and host-memory capacities are the scheduler values displayed by the dashboard [29, 30].	38
4.2	Software configuration declared by the Dockerfile, project meta-data, and current launchers. Version intervals are dependency constraints, not exact installed versions for every experiment.	39
4.3	Episode support in the local corpus. Only the synthetic closing units are removed for the probe experiment.	41
4.4	Benchmark coverage of the correlation corpus. Counts refer to source problems and generated attempts, not to the number of independent observations in every statistical analysis.	42

Chapter 1

Theoretical Framework

The objective of this thesis is to investigate how the reasoning behaviour of a mixture-of-experts (MoE) language model can be guided through interventions on its routing mechanism. This objective connects a behavioural question, concerning the organisation of a generated solution, with a computational question, concerning which experts process its tokens. Establishing that connection requires a vocabulary for describing reasoning, measurements of internal model activity, and a distinction between observing an association and changing the mechanism that produces it.

The project develops four connected stages toward this objective. The first constructs an automatic annotator of reasoning episodes. The second tests whether episode information is accessible from prefix hidden states and selects measurement locations. The third measures associations between routing, representations, reasoning behaviour, and answer correctness. The fourth tests expert-identity and margin policies during free generation. Chapter 3 specifies their integration. This chapter defines the concepts needed to distinguish measurement, association, and the effect of a routing intervention.

1.1 Reasoning as a measurable process

Let q denote a problem and let $x_{1:T}$ be the model’s generated response, including intermediate reasoning and the final answer. An autoregressive

language model with parameters θ defines

$$P_{\theta}(x_{1:T} \mid q) = \prod_{t=1}^T P_{\theta}(x_t \mid q, x_{<t}). \quad (1.1)$$

Each generated token becomes part of the context for later decisions. Intermediate text can therefore affect subsequent computation, even when evaluation ultimately assigns a single correctness label to the final answer. Chain-of-thought prompting makes such intermediate text explicit and has demonstrated benefits on several reasoning benchmarks [1].

For analysis, the response is segmented into units $s_1, \dots, s_J$, usually sentences or short coherent spans. A unit can restate the problem, introduce a strategy, execute a calculation, or check a previous result. These functions provide a richer description than response length or final accuracy alone. However, a verbal explanation is an observable output of the model, rather than a complete record of its internal computation. Experiments on chain-of-thought faithfulness show that a response can omit factors that influenced its answer [2]. Accordingly, this thesis uses the term *reasoning episode* for a function assigned to a text unit, without assuming that the label identifies a unique internal cognitive state.

Three levels of observation will recur throughout the thesis. At the token level, the model produces hidden representations and expert-routing decisions. At the unit level, a segment receives a functional episode label. At the attempt level, the complete response has a final answer and possibly binary indicators of particular reasoning events. Repeated attempts add a fourth level: the source problem. Keeping these levels explicit is necessary both for temporal alignment and for statistical analysis.

1.2 Sparse mixture-of-experts computation

1.2.1 Experts, routers, and conditional computation

A sparse MoE layer contains several parameterised expert networks and a router that selects a subset for each token. The sparsely gated formulation

of Shazeer et al. made it possible to increase model capacity while activating only part of that capacity for an input [3]. In language-model backbones, an MoE module commonly replaces a dense feed-forward sublayer. The expert is therefore a neural transformation within a larger network; it is not an independent model that solves the whole problem.

Let $u_t^{(\ell)} \in \mathbb{R}^d$ be the input to the router at layer ℓ , and let E_ℓ be the number of routed experts there. For a softmax-based router, scores $z_t^{(\ell)}$ define

$$z_t^{(\ell)} = W_r^{(\ell)} u_t^{(\ell)} + b_r^{(\ell)}, \quad (1.2)$$

$$p_{t,e}^{(\ell)} = \frac{\exp(z_{t,e}^{(\ell)})}{\sum_{j=1}^{E_\ell} \exp(z_{t,j}^{(\ell)})}, \quad (1.3)$$

$$S_t^{(\ell)} = \text{TopK}(p_t^{(\ell)}, k_\ell). \quad (1.4)$$

The bias can be absent, and other routing architectures use different score transformations. These equations specify the softmax case used to define the measurements below, rather than a universal implementation contract.

Writing $f_e^{(\ell)}$ for expert e , a routed output has the form

$$m_t^{(\ell)} = \sum_{e \in S_t^{(\ell)}} g_{t,e}^{(\ell)} f_e^{(\ell)}(u_t^{(\ell)}) + m_{t,\text{shared}}^{(\ell)}. \quad (1.5)$$

Here $g_{t,e}^{(\ell)}$ denotes the effective mixing weight, which may be renormalised over the selected experts. The shared contribution is present only in architectures that include shared experts. This distinction matters: the full router distribution p , the selected set S , and the effective weights g are different objects. An intervention on one need not have the same effect as an intervention on another.

Switch Transformer uses one routed expert per token, whereas Mixtral uses two of eight experts [4, 5]. DeepSeekMoE combines finer expert partitioning with shared experts [6]. These architectural choices affect how routing statistics should be interpreted. They also illustrate why total parameter count and active parameter count must be distinguished: sparse activation reduces per-token expert computation, but does not eliminate the cost of storing expert weights or dispatching tokens.

The project’s Qwen3.5-35B-A3B subject provides a concrete instance. Its official model card describes a hybrid backbone with 40 layers, combining Gated DeltaNet and gated attention, with 256 routed experts, eight selected experts per token, and a shared expert [7]. This makes the subject a hybrid sequence model. The general routing notation above applies independently of the particular sequence-mixing sublayer.

1.2.2 Interpreting expert identity

Expert identities are local to a layer. The correct identifier is the pair (ℓ, e) : expert 7 in one layer has different parameters from expert 7 in another layer. Likewise, selecting an expert frequently during a labelled episode does not establish that the expert implements that episode. Token identity, syntax, position, and problem content may all influence selection. Mixtral’s routing analysis, for example, reports syntactic regularities and locality between consecutive tokens [5].

It is useful to distinguish *statistical specialisation*, meaning that expert use differs across observed contexts, from *functional involvement*, meaning that changing the expert’s contribution changes a behaviour of interest. The first can nominate targets for study. The second requires an intervention and an appropriate comparison. Neither implies a one-to-one mapping between experts and semantic reasoning functions.

1.3 A vocabulary for reasoning episodes

1.3.1 Adapting episode theory to model-generated text

Schoenfeld’s account of mathematical problem solving emphasises the organisation and control of problem-solving activity in addition to the availability of mathematical knowledge [8]. Li et al. adapt this perspective to reasoning-model traces and release a corpus with sentence and paragraph annotations [9]. The present project uses seven sentence-level categories. Their operational meaning in the annotation experiment is summarised in Table 1.1.

Table 1.1: Operational episode categories used by the project, based on the adapted framework of Li et al. [9]. The descriptions concern a unit’s function, rather than its mathematical validity.

Episode	Function of the text unit
Read	Restates information or the goal supplied by the problem.
Analyze	Interprets information, introduces structure, or derives a relationship.
Plan	Commits to a concrete operation or strategy before executing it.
Implement	Executes a procedure, calculation, substitution, or enumeration.
Explore	Tentatively proposes a hypothesis, option, or alternative path.
Verify	Checks an existing candidate, result, or consistency claim.
Monitor	Expresses content-light self-regulation, a transition, or a structural boundary.

These labels are nominal categories. They are not ordered from weak to strong reasoning, and a solution need not pass through them in a fixed sequence. A Verify unit can contain an invalid check; an Implement unit can contain an incorrect calculation. Similarly, repeated monitoring does not necessarily improve a solution. Episode analysis and correctness evaluation therefore measure different aspects of model behaviour.

Segmentation is part of this measurement. One sentence may both announce and execute a plan, while a short structural marker may have no mathematical content. Assigning one label requires a documented precedence rule and access to enough context to distinguish, for example, a given fact from a newly derived relation. These ambiguities limit the meaning of exact agreement with a reference annotation. Per-class errors and inspection of ambiguous boundaries are consequently useful alongside aggregate classification scores.

1.3.2 Automatic episode annotation

Let $a_j \in \mathcal{A}$ denote the reference episode for unit s_j , where $\mathcal{A}$ contains the seven categories. A prompted annotator with instruction ϕ returns

$$\hat{a}_j = A_\phi(q, s_{j-1}, s_j, s_{j+1}). \quad (1.6)$$

This is the information pattern used by **gepaLLMAsJudge**: the problem and neighbouring units help identify the current unit’s function. Because the current and next units are visible, it is an offline annotation tool. It cannot be inserted unchanged into a controller that must act before s_j is generated.

GEPA searches over instructions using feedback and natural-language reflection, while keeping the underlying model weights fixed [10]. In this project, a same-model judge can provide structured optimisation feedback, but final prompt selection is assessed against reference labels. Such feedback is not independent ground truth. Splitting complete responses before their units are assigned to training, validation, and test sets prevents neighbouring passages from crossing evaluation boundaries.

Accuracy measures exact label agreement. Balanced accuracy averages recall across categories, and macro-F1 averages the category-specific F1 scores; both expose failures concealed by a dominant class. Cohen’s κ measures chance-adjusted nominal agreement. Rank-based agreement measures require an independently justified ordering and are not primary measures for these episode labels. The purpose of annotation optimisation is to establish a usable measurement instrument for later experiments, rather than to optimise the subject model’s routing directly.

1.4 Prospective representation probes

For a unit beginning at token index b_j , define a boundary representation $v_j^{(\ell)} = h_{b_j-1}^{(\ell)}$. Under causal processing, it depends on the problem and the preceding response, but not on the target unit. A binary linear probe for episode a has the form

$$\hat{P}(a_j = a \mid v_j^{(\ell)}) = \sigma \left(w_{a,\ell}^\top v_j^{(\ell)} + c_{a,\ell} \right), \quad (1.7)$$

where σ is the logistic function. Above-chance prediction indicates that information useful to this classifier is accessible at that boundary. The interpretation depends on probe capacity, train/test separation, and possible shortcuts [11].

The **probeTest** experiment trains a separate one-versus-rest logistic regression for each label and hidden-state index. It teacher-forces released DeepSeek-

R1 responses through a Qwen encoder. Consequently, its immediate subject is Qwen’s representation of those supplied prefixes. It does not observe Qwen generating the annotated solutions. The current replication uses a sentence-level split, so units from a response can appear in both partitions; selecting the best layer on that test split also makes the selected maximum optimistic. A response-grouped evaluation and a separate layer-selection partition are needed to assess generalisation to new traces.

Temporal availability and causal influence must also be distinguished. A pre-unit representation is prospective, but predicting a future label does not establish that the probed direction causes that label. A probe can exploit problem type, position, or formatting. Independently trained, class-balanced binary scores also need not form calibrated multiclass probabilities, even when their largest value is used to propose a label for inspection.

Finally, hidden-state indices must be tied to actual locations in the architecture. Let $h_t^{(\ell)}$ denote a block-input residual state and $u_t^{(\ell)}$ the router input from Section 1.2. An intervening sequence-mixing sublayer and normalisation can make these vectors different. The current correlation extraction retains block-input hidden states alongside the router distribution from the corresponding block; it does not directly capture the exact normalised router-input vector. Geometric relationships between the two measurements must be interpreted with this distinction in mind.

1.5 Routing measurements and statistical analysis

1.5.1 Concentration, selection, and temporal dynamics

For a fixed layer and token, suppress the indices on p , and write $p_{(1)} \geq \dots \geq p_{(E)}$ for the sorted router probabilities. For $E > 1$, Shannon entropy and its normalised concentration transform are

$$H(p) = - \sum_{e=1}^E p_e \log p_e, \quad C(p) = 1 - \frac{H(p)}{\log E}. \quad (1.8)$$

$C = 0$ for a uniform distribution and approaches one as mass concentrates on one expert. Although the implementation calls C routing confidence, it is not a calibrated probability that the answer is correct. Routing probabilities describe expert allocation; vocabulary probabilities describe the next token. Their entropies concern different distributions.

For top- k routing with $k < E$, additional measures are

$$M_k(p) = \sum_{i=1}^k p_{(i)}, \quad (1.9)$$

$$B_k(p) = p_{(k)} - p_{(k+1)}, \quad (1.10)$$

$$G_k(p) = \frac{M_k(p)}{k} - \frac{1 - M_k(p)}{E - k}. \quad (1.11)$$

Selected mass M_k measures how much probability is assigned to the active set. The boundary margin B_k describes the separation between the last selected and first unselected expert. G_k compares mean selected and unselected probability. The top-1/top-2 margin alone does not describe the selection boundary when more than one expert is active.

Some of these quantities contain redundant information. For fixed E and k , G_k is an affine function of M_k ; their associations cannot be treated as independent evidence. Similarly, C is an affine sign reversal of H . Presenting a transformed measure can aid interpretation, but does not create an additional empirical finding.

For a trace of $T > 1$ tokens, the current implementation summarises temporal routing with top-1 switching and top- k set overlap:

$$W^{(\ell)} = \frac{1}{T-1} \sum_{t=2}^T \mathbf{1} \left[\arg \max_e p_{t,e}^{(\ell)} \neq \arg \max_e p_{t-1,e}^{(\ell)} \right], \quad (1.12)$$

$$O^{(\ell)} = \frac{1}{T-1} \sum_{t=2}^T \frac{|S_{t-1}^{(\ell)} \cap S_t^{(\ell)}|}{k_\ell}. \quad (1.13)$$

The overlap denominator is the number of selected experts, not the union size used by Jaccard similarity. High overlap indicates persistent selection, but its relation to useful reasoning is an empirical question.

Identity features can instead measure the proportion of tokens selecting a particular (ℓ, e) , or the frequency of a particular expert pair. An ordered

top-1/top-2 pair describes co-selection at one token; it does not describe a transition between tokens or every combination in a top- k set. Layer identity must be preserved before such features can nominate individual experts for intervention.

Hidden-state summaries provide complementary measurements, including vector norm and consecutive-token cosine distance,

$$D^{(\ell)} = \frac{1}{T-1} \sum_{t=2}^T \left(1 - \frac{\langle h_t^{(\ell)}, h_{t-1}^{(\ell)} \rangle}{\|h_t^{(\ell)}\| \|h_{t-1}^{(\ell)}\|} \right). \quad (1.14)$$

The pipeline also constructs a geometry feature by correlating pairwise cosine similarities among sampled hidden states with the corresponding similarities among router distributions. That internal construction uses Pearson correlation. It is a scalar descriptive feature, distinct from the coefficient subsequently used to associate it with correctness; the many token pairs are not treated as independent experimental observations.

1.5.2 Binary correctness and repeated-attempt accuracy

Let $Y_{ia} \in \{0, 1\}$ indicate whether attempt a on problem i produces a correct final answer, and let X_{ia} be a routing or representation feature. Their attempt-level association is measured by point-biserial correlation,

$$r_{\text{pb}} = \text{Corr}(X, Y). \quad (1.15)$$

This is exactly Pearson correlation when correctness is encoded as zero or one. A positive coefficient means that the feature tends to be larger among correct attempts. Its magnitude can depend on class prevalence and feature distributions; it is neither a causal effect nor a measure of probability calibration. The same coefficient applies to a continuous feature paired with a binary reasoning-event indicator. Liu et al. provide direct language model precedent for this continuous-versus-binary setting [12].

With N attempts per problem, define

$$\widehat{A}_i^{(N)} = \frac{1}{N} \sum_{a=1}^N Y_{ia}, \quad \overline{X}_i = \frac{1}{N} \sum_{a=1}^N X_{ia}. \quad (1.16)$$

The first quantity is the empirical per-problem $\text{avg}@N$. For $N = 32$, it takes values in $\{0, 1/32, \dots, 1\}$. It differs from $\text{pass}@N$, which concerns at least one successful attempt among N samples. The primary repeated-attempt analysis uses one pair $(\bar{X}_i, \hat{A}_i^{(N)})$ per problem and computes Spearman correlation,

$$\rho_s = \text{Corr}\left(\text{rank}(\bar{X}_i), \text{rank}(\hat{A}_i^{(N)})\right), \quad (1.17)$$

using average ranks for ties. The scientific question is whether larger feature values tend to accompany larger success fractions, without requiring a linear relation between their numerical scales. Boundedness alone does not make Pearson invalid; Spearman is selected because monotone association is the target. Small problem counts and many tied success fractions still limit precision. Section 2.6 discusses the supporting literature.

Complete groups are necessary to make $\text{avg}@N$ comparable across problems. The analysis checks declared sample identifiers, outcome availability, and feature coverage, and audits exact duplicate generations where their fingerprints are available. A known gold answer with no parseable generated answer is counted as incorrect; a task without a scoring target remains unscored. Truncation and answer-parsing failures must be reported because they can alter both trace features and the measured success fraction.

1.5.3 Dependence, uncertainty, and confounding

Attempts on the same problem share difficulty and content. Counting them as independent observations can make uncertainty appear smaller than it is. For prespecified aggregate features, the pipeline therefore estimates 95% percentile bootstrap intervals by resampling whole problems and retaining their attempts together. For problem-level Spearman coefficients, it resamples the observed problem pairs. This latter procedure quantifies variation across observed problems without separately modelling uncertainty from the finite number of attempts within each problem.

An additional descriptive comparison uses only problems that yield both correct and incorrect attempts:

$$\Delta_i = \bar{X}_{i,Y=1} - \bar{X}_{i,Y=0}. \quad (1.18)$$

Averaging these contrasts removes fixed differences between source problems, but still permits differences in response length, token composition, or generation history within a problem. It also describes the subset of mixed-outcome problems. It is not an estimate of what would happen if routing were changed. The project’s GPQA design varies answer-option positions between attempts, so it is excluded from this particular contrast.

Searching many layers, features, experts, and pairs creates a multiplicity problem. Benjamini–Hochberg adjustments provide a declared way to summarise multiple tests within a specified family [13]. They rely on valid input tests and appropriate dependence assumptions. In particular, applying the adjustment to naive attempt-level p -values does not repair their failure to account for repeated problems. Such values remain exploratory diagnostics. A confirmatory intervention should specify its outcomes, targets, and evaluation partitions independently of these scans.

1.6 From observations to routing intervention

A routing intervention modifies the computation in Equations 1.4–1.5 during inference. One possible family of policies adds a context-dependent bias to the router scores:

$$\tilde{z}_t^{(\ell)} = z_t^{(\ell)} + \alpha v^{(\ell)}(q, x_{<t}), \quad (1.19)$$

after which expert selection and mixing weights are recomputed. This equation describes a design space for the thesis, rather than an already validated implementation. The strength α , affected layers, target experts, and activation times are parts of the policy.

Alternative policies reweight experts already selected, mask particular experts, or alter the selected set. These choices test different hypotheses. For example, rescaling all logits by a positive temperature changes their concentration but preserves their ordering and therefore preserves a deterministic top- k set. Increasing the contribution of an already selected expert also differs from forcing an otherwise unselected expert to participate. In all cases, renormalisation and the contribution of shared experts must be specified to interpret the

intervention.

A controller intended to act before a reasoning unit can use only information available at that time. Offline labels may help construct or evaluate that controller, but the target unit and its future continuation cannot supply its decision at deployment. Prospective probes are relevant because they test whether useful information exists under this restriction. Their predictions still require validation on the generation distribution where the controller will operate.

Teacher-forced replay is useful for measuring activations on a fixed saved response, but an intervention during free generation changes subsequent tokens and therefore subsequent states. A replay-based change in a feature or next-token score is not sufficient evidence of improved problem solving. The intervention objective ultimately concerns a contrast such as

$$\Delta_\pi = \mathbb{E}_{q,\xi} [Y(q, \xi; \pi) - Y(q, \xi; \pi_0)] , \quad (1.20)$$

where π_0 is the unmodified router, π is the intervention, and ξ represents sampling randomness under a specified evaluation protocol. Episode frequencies, answer correctness, and computational cost should be measured together: increasing the amount of checking language is not itself evidence of better reasoning.

This formulation motivates four connected research questions. Can the episode vocabulary be applied reliably to new traces? Can relevant behaviours be detected from information available before they occur? Which routing measurements remain associated with behaviour and correctness after accounting for major confounders? Finally, does acting on selected routing targets improve outcomes on held-out problems under a controlled generation budget? The annotation, probe, and correlation stages address the measurement and observational questions; the guiding experiments specified in Chapter 3 address the effect of intervening during generation.

Chapter 2

Related Work

Research relevant to routing-guided reasoning spans several levels of model behaviour. Some approaches change the prompts or training signals that produce reasoning traces. Others annotate those traces, decode information from internal representations, or directly modify activations and expert routing. This chapter compares those approaches according to the object they measure or manipulate, and positions the four-stage experimental pipeline within that literature.

2.1 Reasoning traces, evaluation, and functional annotation

Chain-of-thought prompting demonstrated that intermediate reasoning demonstrations can improve language-model performance on arithmetic, commonsense, and symbolic tasks [1]. This established generated reasoning text as a practical object of study, but it did not make final accuracy a complete measure of the quality of a reasoning process. Lightman et al. distinguish outcome supervision from feedback on intermediate step validity and release PRM800K for studying process supervision [14]. Their distinction is relevant to this thesis because a correct final answer and a well-formed intermediate process are related but separate evaluation targets.

Functional episode annotation addresses a further question: what a step

is doing within the solution. Building on Schoenfeld’s framework, Li et al. annotate model-generated mathematical solutions at sentence and paragraph levels and analyse transitions between episode categories [9]. Their sentence-level labels supply the project’s annotation vocabulary and released reference data. A semantic episode differs from a process-reward label: Verify describes an attempt to check something, whereas a validity label evaluates whether a step is correct. The same functional category can therefore occur in both successful and failed traces.

The faithfulness experiments of Turpin et al. further limit what can be inferred from generated explanations [2]. They show settings where input cues influence a prediction without being acknowledged in the accompanying rationale. For the present project, the implication is methodological: an episode sequence is an informative description of generated text, but a claim about the mechanism producing it needs independent internal evidence and intervention.

SPIRAL studies reasoning through a different route, using self-play on zero-sum games as a reinforcement-learning training signal [15]. Its evaluation includes mathematical and knowledge-intensive benchmarks also used by the correlation pipeline, and it analyses textual reasoning patterns. The project adopts this benchmark suite with its own benchmark-specific repeated-attempt protocol; it does not claim to reproduce every evaluation setting of the SPIRAL paper. Self-play changes the training process, whereas the thesis’s intended control mechanism acts on routing during inference.

2.2 Prompt optimisation for episode annotation

An automatic episode annotator must handle closely related categories, class imbalance, and contextual ambiguity. GEPA addresses the broader problem of optimising language-model systems through reflective prompt evolution [10]. It uses execution and evaluation traces to generate natural-language feedback, proposes prompt revisions, and maintains candidates with complementary strengths through Pareto-based selection. The optimisation operates with fixed

model weights.

The `gepaLLMAsJudge` experiment applies this approach to the seven-label annotation task. It supplies the problem and neighbouring text units, evaluates predictions against the released annotations, and preserves response boundaries in its data splits. A same-model judge can supply detailed feedback during the search, while agreement with the reference labels and per-class recall constraints govern final selection. These are properties of the project’s experimental design, rather than claims about GEPA’s original evaluation.

The distinction between optimisation feedback and final evaluation is central. A candidate selected after many comparisons on the same validation responses can exploit peculiarities of that partition. Furthermore, a judge that shares the classifier’s model may share its errors. Response-grouped evaluation and an untouched test partition address a different question from whether the search found a better validation prompt. The resulting prompt is intended as a frozen annotation instrument for subsequent experiments; its offline context and reference-label provenance must remain visible when its labels are used to study routing.

2.3 Probing representations of reasoning

Probing asks whether a chosen decoder can extract a property from a model’s representations. Hewitt and Manning demonstrate that a learned transformation of contextual word representations can recover aspects of syntactic tree geometry [16]. Hewitt and Liang subsequently show how control tasks help distinguish information in representations from a probe’s capacity to learn the target task [11]. Together, these works motivate simple, inspectable probes and evaluation designs that make shortcut learning visible. Their linguistic tasks do not establish that reasoning episodes are encoded in the same way; that is an empirical question for this project.

Sun et al. study reasoning as trajectories through representation space [17]. Their analysis examines step-specific geometry and correctness signals in a Llama family, and they investigate adaptive activation changes toward reference trajectories. Their step categories describe positions in a sequence, rather than

Schoenfeld semantic functions. This distinction makes their layerwise probing protocol relevant without making the two annotation tasks equivalent. They also report more limited transfer for correctness prediction than for step-related structure.

The **probeTest** experiment combines a boundary-probe design with the released semantic episode annotations. It extracts representations immediately before the target unit, which creates a prospective prediction task. Its current sentence-split replication is useful for comparison with the adopted protocol, but shared response identities and test-based layer selection limit generalisation claims. Moreover, applying the probes to supplied reference solutions tests representation transfer under teacher forcing. Using them to control the model’s own generation requires additional validation.

Chen et al.’s recent preprint, *Beyond the Trace*, connects internal readouts directly to native MoE routing [18]. They construct J64, a 64-axis semantic frame from reasoning states, and learn a routing-based proxy, R64, from expert usage and gate entropy. They evaluate candidate selection, online stopping and resampling, and targeted router edits. This is a close precedent for connecting process interpretation to routing and test-time decisions. The distinction here is the target vocabulary: the present project starts from externally defined Schoenfeld episodes, whereas J64 is derived from model states. Reconstructing a latent readout, predicting an upcoming episode, and improving an answer through intervention remain different evaluation tasks. Their question-grouped evaluation and frozen cross-benchmark transfer also provide useful methodological comparisons.

This line of work supplies possible detection signals and candidate layers. It does not identify a router intervention by itself. A decoder may read information from a representation that is distributed across many experts, or from a component that the model does not use in the way the decoder does. The transition from decodability to controllability therefore remains a separate experimental step.

2.4 Expert specialisation and direct routing control

Early sparse MoE work establishes conditional computation as a way to scale capacity while limiting active computation [3]. Switch Transformer, Mixtral, and DeepSeekMoE explore different selection and expert-sharing designs [4, 5, 6]. These studies support the architectural motivation for treating routing as a control point. They also caution against assuming that its units already correspond to human-readable reasoning functions. In particular, Mixtral’s analysis of syntactic routing and token locality makes lexical controls relevant when comparing experts across reasoning episodes.

Olson et al. provide controlled evidence of semantic routing [19]. They compare expert overlap when a fixed word changes sense across contexts and when a fixed context contains semantically similar or dissimilar replacement words. These controls make their study relevant to separating meaning from lexical identity. However, word-sense sensitivity does not establish that experts specialise in multi-sentence reasoning episodes. For the present analysis, it motivates checking whether episode–routing associations persist beyond differences in the words used to express each episode.

Ye et al.’s preprint shifts the unit of interpretation from individual experts to paths across layers [20]. Their decomposition separates a routing control component from an orthogonal component invisible to the router. Across the architectures studied, they report that individual experts remain polysemantic while expert trajectories group tokens by semantic function across surface forms. This suggests a limitation of per-expert episode correlations: an expert can participate in several functions, with specificity emerging through its combination with other layers. Path-level analysis is consequently a possible extension of the present measurements, rather than a property demonstrated by them.

RICE is a close precedent for reasoning-directed expert intervention [21]. Wang et al. associate experts with reasoning-related marker tokens using normalised pointwise mutual information, then reinforce identified experts during inference. Their reinforcement multiplies an expert’s mixture weight

when that expert is already in the selected set; it does not necessarily make an unselected expert participate. The paper’s description of cognitive experts is a working hypothesis. Reported sensitivity to reinforcement strength and normalisation, together with limited sampled transfer in one AIME comparison, makes clear that observed specialisation does not automatically imply a robust intervention benefit.

SteerMoE investigates behaviour-associated expert activation using contrasting inputs, and promotes or suppresses experts by modifying routing scores before top- k selection [22]. Its experiments concern behaviours such as document faithfulness and safety. This provides direct evidence that expert selection can be used as an inference-time control mechanism for the studied settings. Its behavioural targets differ from mathematical episode structure, so the transfer to reasoning-guided routing remains a question. The difference from RICE is also operational: changing scores before selection can change membership of the active set, whereas reinforcing an already selected expert primarily changes its contribution.

Yoon et al. examine whether the trained router selects favourable expert combinations by comparing standard routes with sampled alternatives at equal expert compute [23]. Their counterfactual analysis uses successful continuations and also studies a final-router update. This is relevant evidence that routing quality can be investigated separately from expert capacity. However, a next-token score on a supplied continuation is a proxy for free-generation success, and layerwise route comparisons do not by themselves identify an episode-level intervention effect.

Su et al. study continuous test-time rerouting under deployment shifts [24]. Their method optimises additive router-logit vectors in selected layers during prefill and at subsequent intervals, using the available context and generated sequence for self-supervision. Generation then continues with the adapted routing until the next update. This offers a comparison with interventions whose expert targets are identified offline: the routing adjustment is optimised online, without external reference data. Its adaptation objective is not an explicit episode vocabulary, so it does not directly test episode-guided control. Comparisons would also need to account for the extra optimisation cost at

inference time.

These studies establish that routing control is an existing research direction. The question developed here is more specific: whether a validated semantic episode vocabulary, prospective internal signals, and outcome-associated routing measurements can jointly support interventions that improve reasoning on held-out problems. Demonstrating that conjunction requires measurements and evaluations beyond finding experts associated with a few textual markers.

2.5 Activation steering

Routing edits should also be compared with methods that act directly on representations. Contrastive Activation Addition constructs steering directions from differences between activations on contrasting examples and adds those directions during generation [25]. This changes the residual representation rather than explicitly specifying which experts are selected. It is consequently a useful conceptual baseline for separating the value of a behavioural direction from the value of acting at the router.

Hong et al. study a residual-stream direction associated with the balance between reasoning and memorisation [26]. Their analysis relates scalar projections to graded reasoning scores and includes interventions on the corresponding features. Sun et al.’s trajectory work similarly moves beyond representation analysis to interventions that use reference reasoning trajectories [17]. These papers show why identifying a direction and testing its manipulation belong to distinct stages of a causal investigation.

For an MoE subject, a residual edit may indirectly alter routing as well as other downstream computations. Conversely, a router edit changes the mixture of expert outputs and therefore the residual trajectory that follows. The two intervention families are connected, but their direct targets differ. A routing-focused thesis must specify that target and evaluate its effects under comparable generation conditions. A change in episode labels alone would leave open whether the method improved problem solving or only changed how the response was expressed.

2.6 Statistical evidence for metric selection

The choice of correlation coefficient follows the outcome scale and the scientific question. Liu et al. use point-biserial correlation between continuous quantities, including context length and answer probability, and binary correctness in figurative-language interpretation [12]. This is direct precedent for the attempt-level continuous-feature/0–1-outcome setting. The additional justification is algebraic: point-biserial is Pearson correlation under binary coding, with an interpretable direction given by the group difference.

For graded targets, Hong et al. report Spearman association between residual-stream projections and model-assigned reasoning scores [26]. Hewitt and Manning use rank correlation in evaluating representation-derived syntactic geometry [16]. Xu et al. relate language-model-derived conceptual representations to human behavioural ratings using Spearman correlation and employ bootstrap confidence intervals [27]. These are precedents for rank-based analysis of internal quantities, rather than direct studies of avg@32 correctness. The project’s choice of Spearman for per-problem mean correctness is therefore justified by its monotone association target, not by claiming that these papers uniquely prescribe that coefficient.

Zhang et al. report both Pearson and Spearman correlations when comparing uncertainty-based hallucination scores with graded human judgements [28]. Their use of entropy-related quantities is relevant to this project’s concentration features, although token uncertainty and expert-routing entropy refer to different distributions. Reporting both coefficients can answer distinct linear and monotone questions; it does not turn two analyses of the same data into independent replications.

For the correlation experiment, the sample hierarchy matters as much as the coefficient. Thirty-two attempts from a single problem provide repeated observations of that problem, rather than 32 independent problems. The problem-level analysis and cluster bootstrap described in Section 1.5 address that hierarchy. Xu et al.’s use of 10,000 bootstrap resamples in one analysis provides a precedent for reporting resampling uncertainty, but does not determine an appropriate resample count or sampling unit for a different experiment

[27].

Benjamini and Hochberg’s multiple-testing framework motivates declaring families of feature, layer, and expert comparisons [13]. Multiplicity adjustment cannot remove confounding, repair non-independent tests, or turn a discovered association into a causal effect. The active analysis therefore serves to prioritise hypotheses whose intervention effects must subsequently be tested on separate data.

2.7 Position of the experimental pipeline

Table 2.1 relates the active experiments to the evidence required by the thesis objective. Their contributions are complementary: a reliable annotation instrument makes behavioural comparisons possible; a prospective probe tests whether suitable signals are available before a decision; a routing analysis narrows the set of mechanisms worth changing; and guiding experiments evaluate those changes during generation.

The completed method connects these stages without requiring an episode-aware controller. The frozen annotator supplies offline behavioural labels, and the probes select measurement locations. Identity and margin policies are then calibrated from routing measurements and correctness, independently of episode labels. This design avoids relying on the transfer accuracy of the probes as online classifiers. Their layer selection remains exploratory under the replicated split and selection protocol.

Generation and measurement provenance impose a further interpretative limit. Quantised generation and replay implementations can differ, so replay measurements need not equal the generator’s internal activations. Guiding policies retain that provenance and are evaluated by changing the actual router used for free generation. Expert-identity calibration also preserves the full (ℓ, e) pair, avoiding the ambiguity of pooled screens that pooled equal numeric expert indices across layers.

The contribution pursued by the thesis is an experimentally grounded connection between semantic reasoning behaviour and controllable sparse computation. Chapter 3 specifies this connection through the four-stage pipeline.

Table 2.1: Role of each stage in the routing-intervention pipeline. These are research functions, rather than claims of completed causal validation.

Experiment	Evidence sought	Role in intervention research
<code>gepaLLMAsJudge</code>	Agreement between automatic episode labels and reference annotations on separated responses.	Defines a reproducible behavioural measurement instrument.
<code>probeTest</code>	Predictability of upcoming episodes from pre-unit hidden states.	Nominates signals and locations for detecting relevant contexts during generation.
<code>correlation_pipeline</code>	Associations of routing and hidden-state features with correctness and reasoning events.	Nominates routing properties and expert-use patterns for controlled tests.
Routing guidance	Paired changes in correctness and generated-token counts on held-out problems.	Tests calibrated expert-identity biases and margin targets during free generation.

Its empirical success is assessed by the effect of frozen routing policies on held-out outcomes and token use.

Chapter 3

Method

The method connects semantic descriptions of reasoning to controlled changes in sparse expert computation. It consists of four stages: optimising an LLM-based episode annotator with GEPA, identifying informative hidden-state locations with probes, measuring associations between routing and reasoning outcomes, and testing routing policies during generation. Model parameters remain frozen throughout. The learned objects are an annotation prompt, linear classifiers, and calibration-derived routing policies.

These stages have distinct roles. The annotator describes the function of text already produced; the probes identify locations where information about an upcoming episode is accessible; the correlation experiment characterises routing on generated solutions; and the guiding experiments test whether changing routing changes performance. Figure 3.1 shows the flow of information. Neither guiding policy requires a predicted episode label at inference time. Episode annotation supplies a behavioural analysis, while the deployed interventions use expert identity or the current router margin.

3.1 Optimising the LLM-as-judge prompt

The first stage operationalises the seven reasoning episodes defined in Section 1.3: Read, Analyze, Plan, Implement, Explore, Verify, and Monitor.

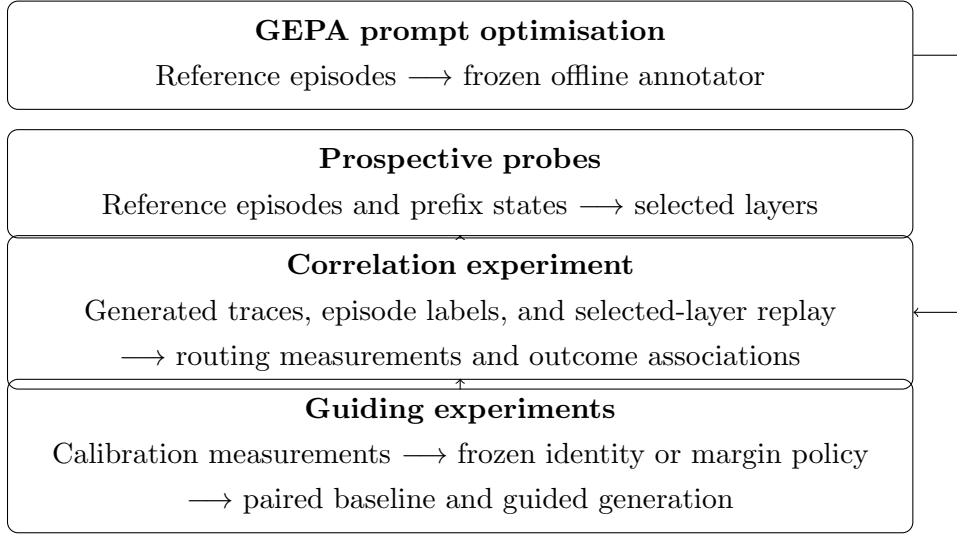


Figure 3.1: Information flow through the method. Annotation and probing use the reference corpus independently. Their outputs meet in the correlation experiment; guiding policies are fitted using calibration problems only.

For problem q and response unit u_j , the annotator receives the context

$$c_j = (q, u_{j-1}, u_j, u_{j+1}), \quad \hat{y}_j = A_\phi(c_j), \quad (3.1)$$

where ϕ is the instruction prompt. The model assigns exactly one label to the current unit. Neighbouring units help distinguish, for example, announcing an operation from performing it, or deriving a result from checking an existing result. The problem statement is visible, but reference answers and rationales are excluded. Since the next unit is available, this is an offline measurement procedure.

GEPA searches over ϕ using performance feedback while leaving the annotator’s weights fixed. The initial context-aware prompt contains the category definitions, boundary rules, and 21 audited synthetic examples, three per category. Complete responses are partitioned before their units are flattened so that neighbouring units cannot cross the annotation training and validation boundaries. Configuration comparison uses nested response-grouped cross-validation; the selected configuration is then fitted to obtain a deployable prompt. The concrete partitions and model settings are specified in Section 4.4.

Two uses of the language model must be distinguished. The episode annotator predicts $\hat{y}_j$ without access to the reference label. During prompt optimisation, a separate call to the same model evaluates that prediction with access to the reference label. This feedback judge supplies an explanation and four scores $s_d \in \{1, \dots, 5\}$: gold agreement, functional fit, contextual coherence, and boundary precision. The reward is

$$R(c_j, \hat{y}_j, y_j) = \sum_{d=1}^4 \omega_d \frac{s_d - 1}{4}, \quad (\omega_1, \omega_2, \omega_3, \omega_4) = (0.40, 0.25, 0.15, 0.20). \quad (3.2)$$

The resulting score lies in $[0, 1]$. The explanations provide information about category confusions for prompt revision. Exact-match and inverse-frequency weighted rewards provide alternative optimisation conditions.

The feedback score is not the final prompt-selection criterion. Selection compares the candidate and seed prompts against reference annotations, using balanced accuracy by default. A candidate is retained only if it improves that measure and no class recall decreases by more than 0.10. Otherwise, the seed remains the selected prompt. This distinction matters because a same-model judge can reinforce its own preferences; agreement with its feedback is not an independent estimate of annotation accuracy. The later Qwen3.8 alignment retained the imported prompt under this selection rule.

The selected instruction is frozen before downstream tagging. Each generated reasoning unit is assigned a label using the same context construction, and its text span is aligned to completion tokens. Unlabelled units remain unlabelled rather than being imputed from probe predictions. Classification quality is assessed through accuracy, balanced accuracy, macro-F1, per-category precision and recall, and valid-output coverage. Changes to the serving check-point or reasoning configuration are recorded because a frozen prompt alone does not imply unchanged annotation behaviour.

3.2 Identifying relevant layers with prospective probes

The second stage identifies hidden-state locations informative about the next reasoning episode. It uses the reference annotations directly, without substituting the automatic judge’s labels. The released problem and response are supplied to each frozen subject model through teacher forcing. For a unit whose first token has position b_j , the feature at hidden-state index ℓ is

$$v_j^{(\ell)} = h_{b_j-1}^{(\ell)}. \quad (3.3)$$

This boundary excludes the target unit and all later tokens. Consequently, the probe measures information available in the prefix before the unit is read, rather than classifying its text retrospectively. Synthetic closing `</think>` units are removed to avoid learning a structural shortcut.

For each episode c and hidden-state index ℓ , a separate logistic regression predicts the binary target $\mathbb{I}[y_j = c]$:

$$P_{c,\ell}(v_j) = \sigma\left(w_{c,\ell}^\top v_j^{(\ell)} + b_{c,\ell}\right). \quad (3.4)$$

The replication uses raw activations, balanced class weights, and L2 regularisation. The sentence-level split and classifier settings are held constant across models. Accuracy, F1, and AUROC are computed at every index. For each category, the saved protocol selects the index with greatest test accuracy, resolving ties toward the earlier index.

The output used by the correlation experiment is the union of those selected indices that correspond to actual router layers. Table 3.1 records the resulting measurement locations. Hidden-state index zero denotes the embedding output; an intermediate index denotes the state entering the decoder block with that index. The final output index has no following router. Indices are therefore filtered for router availability without shifting or renumbering them. In particular, Qwen’s Explore selection at hidden-state index 40 is omitted from routing extraction.

Layer selection narrows the measurement space but does not establish causal importance. Units from the same response occur on both sides of the probe

Table 3.1: Probe-derived router locations used by the method. Indices are zero-based decoder indices and are specific to each model’s probe run.

Subject model	Retained router indices
Qwen3.5-35B-A3B	27, 34, 35, 36, 38, 39
GPT-OSS-20B	15, 19, 21, 23
Gemma-4-26B-A4B	16, 19, 21, 22, 25, 27, 29

split, and the same test partition is used for selection and evaluation. The selected scores are therefore exploratory measures of within-corpus decodability. Moreover, a block-input state is not necessarily the vector presented directly to its router after sequence mixing and normalisation. The subsequent experiments preserve this distinction. They use the selected locations for routing measurements, without treating the probes as validated online episode detectors or optimal intervention selectors.

3.3 Relating routing to reasoning and correctness

3.3.1 Generation, annotation, and replay

The correlation stage begins with freely generated benchmark solutions. Each record identifies the dataset, source problem, attempt, generation settings, response, final-answer score, and termination status. Multiple attempts share a source-problem identifier. Correctness is defined by the benchmark answer checker; an unparseable answer to a scored problem is incorrect, while a genuinely unscored task has a missing outcome.

Generation and internal measurement are separate operations. The saved continuation is replayed through the corresponding instrumented model to capture hidden states and router behaviour at the retained layers. The current replay path uses the original prompt and completion token IDs, together with tokenizer fingerprints and character offsets, rather than reconstructing a

response from rendered text. This preserves native chat and reasoning markers and permits sentence labels to be associated with the same token positions used for feature extraction.

Generation and replay can use different quantised implementations of a checkpoint lineage. Replay measurements therefore describe the replay model on the saved continuation. Numerical equivalence with the generator is not assumed. The same provenance is carried into policy calibration: transferring an expert identity or a numeric margin range between implementations is an explicit experimental assumption.

The frozen annotator supplies class labels for the tagged analysis. When annotation is restricted to a sampled population, the sampling manifest specifies which attempts and sentences are covered. In particular, the standard tagging path selects attempt zero and allocates a sentence budget across benchmarks according to observed error rates. Such class statistics describe that selected population. Aggregate routing analyses and analyses of available tagged units retain their own denominators; missing labels do not imply an absence of the corresponding episode. Untagged runs also support separate lexical indicators of backtracking, contradiction, and self-correction, which are not interchangeable with the seven episode labels.

3.3.2 Measurements and analysis views

Let $p_{i,t,e}^{(\ell)}$ be the full router probability for expert e at token t of attempt i , and let $S_{i,t}^{(\ell)}$ be the selected expert set. The method measures concentration, probability margins, selected mass, selection-boundary gaps, and changes in expert selection across successive tokens. Hidden-state norms, successive-state distances, and hidden/router geometry provide complementary representation measurements. Their definitions are given in Chapter 1. Expert membership is measured anywhere in the native top- k set, not only at rank one. Every intervention target retains the full pair (ℓ, e) , since equal expert numbers at different layers identify different modules.

Features are summarised over the whole continuation, the reasoning span, each available episode class, and absolute-position windows. The position view

uses common token boundaries derived from the corpus mean reasoning length, with an overflow window. A short response contributes only to windows it reaches. Class views similarly require observed labelled tokens of the class. Coverage accompanies each estimate so that changes in the analysed population remain visible. Full distributions need not be retained for all analyses: compact scalar summaries and selected expert tensors preserve the measurements required for correlation and guiding calibration.

3.3.3 Statistical unit and outcome association

For a continuous feature f_i and binary correctness Y_i , the attempt-level association is the point-biserial correlation. Positive values mean that the feature is larger in correct attempts. The same convention applies to binary reasoning-event indicators. For repeated-attempt benchmarks, a separate problem-level analysis relates the mean feature to mean correctness using Spearman correlation. With n attempts, this outcome is $\text{avg}@n$, not the probability that at least one attempt succeeds. The primary problem-level analysis requires the complete declared set of unique attempts and complete binary outcomes. Problems containing both correct and incorrect attempts also permit within-problem feature contrasts.

Repeated attempts are not independent observations. For prespecified aggregate associations, uncertainty is estimated by resampling source problems and retaining their attempts together; the attempt-level bootstrap is stratified by dataset. Problem-level Spearman intervals resample complete problem summaries. Benjamini–Hochberg adjustment is applied within the declared testing families. Broad layer, expert, and feature-pair screens remain exploratory, and conventional attempt-level significance tests do not replace uncertainty estimates that account for problem clustering.

The current analysis retains token-limit completions and reports their termination status in a generation-budget audit. Missing or nonfinite measurements are omitted for the relevant feature–outcome pair. This keeps the target population distinct from a completed-response-only analysis. Length, benchmark difficulty, and episode composition can affect both routing and correctness,

so an observed association is a candidate for intervention, not evidence that changing the feature will improve an answer.

3.4 Guiding experiments

The final stage tests two policies separately: promoting expert identities associated with correctness and moving router margins toward ranges associated with correctness. Both act on native router outputs during free generation, with expert network parameters unchanged. They preserve the model’s expert count, native top- k dispatch, and shared computation. Qwen uses eight of 256 routed experts, GPT-OSS four of 32, and Gemma eight of 128. Policies are fitted separately for each model and cannot exchange expert identities across unrelated checkpoints.

3.4.1 Calibration and held-out evaluation

The global workflow partitions source problems within each benchmark into 70% calibration and 30% evaluation using seed 42. All attempts of a problem remain on the same side. Calibration consumes scored traces with the required routing measurements and records exclusions. Evaluation uses the held-out generation problems irrespective of whether their earlier traces had routing tensors or usable scores. Both correct and incorrect calibration answers are required to estimate an accuracy advantage.

For calibration trace i belonging to problem $q(i)$, let $n_{q(i)}$ be the number of available calibration attempts for that problem and define $w_i = 1/n_{q(i)}$. Each problem then has total weight one. The common baseline for policy fitting is

$$A_0 = \frac{\sum_i w_i Y_i}{\sum_i w_i}. \quad (3.5)$$

This weighting prevents repeated-attempt benchmarks from dominating solely through their number of attempts; datasets with more problems still have more calibration weight. Policies are frozen before evaluation. Any tuning of intervention strength, support thresholds, or binning must use a separate validation partition rather than held-out outcomes.

3.4.2 Expert-identity guiding

Identity guiding uses the frequency with which an expert belongs to the selected pool. For a trace with T_i saved completion positions, define

$$r_{i,\ell,e} = \frac{1}{T_i} \sum_{t=1}^{T_i} \mathbb{1}[e \in S_{i,t}^{(\ell)}], \quad A_{\ell,e} = \frac{\sum_i w_i r_{i,\ell,e} Y_i}{\sum_i w_i r_{i,\ell,e}}. \quad (3.6)$$

The first normalisation prevents long traces from dominating through their number of tokens. $A_{\ell,e}$ is a frequency-weighted correctness estimate, not the success rate of an expert acting in isolation. Its lift is $L_{\ell,e} = A_{\ell,e} - A_0$.

At each layer, experts are eligible if they have positive lift and occur in at least four distinct calibration problems. At most eight are retained, ordered by lift, then problem support, then expert identifier. Retained lifts are divided by the largest retained lift at that layer to produce scores $v_{\ell,e} \in (0, 1]$; other scores are zero. Layers without eligible experts receive zero scores, and calibration fails if no layer has an eligible target.

During generation, the policy adds a fixed bias to each selected layer’s router logits:

$$\tilde{z}_{t,e}^{(\ell)} = z_{t,e}^{(\ell)} + \alpha v_{\ell,e}, \quad \alpha \geq 0. \quad (3.7)$$

Native dispatch then selects and weights experts from the adjusted logits. The bias can change both membership and mixture weights. The policy is independent of episode labels and does not wait for a particular margin or reasoning event. Its purpose is to test whether the calibrated association survives an intervention that increases the influence of those identities.

3.4.3 Margin guiding

Margin guiding instead targets a distributional property without assigning preferred expert identities. Let $p_{(r)}$ denote the probability at descending rank r before selected-expert renormalisation. The default metric is the selection-boundary margin

$$m = p_{(k)} - p_{(k+1)}. \quad (3.8)$$

The top-1/top-2 gap is an alternative configuration. For each layer, calibration partitions trace-mean margins into up to five problem-weighted quantile bins,

keeping ties together. Among bins supported by at least four distinct problems and with accuracy above A_0 , it selects the bin with highest weighted accuracy, breaking ties by support and then lower margin. The observed minimum and maximum define the frozen interval $[a_\ell, b_\ell]$. Layers without an eligible bin are skipped; a policy with no eligible layer is rejected.

At a live token, the desired margin interpolates toward the nearest point of this interval:

$$d = m + \lambda(\text{clip}(m, a_\ell, b_\ell) - m), \quad 0 \leq \lambda \leq 1. \quad (3.9)$$

Let $r = k$ for the boundary margin, or $r = 1$ for the alternative gap, and let E be the number of routed experts. To decrease the margin, the policy mixes the full probability vector with the uniform distribution u_E :

$$\tilde{p} = (1 - \beta)p + \beta u_E, \quad \beta = \frac{m - d}{m} \quad (d < m). \quad (3.10)$$

To increase it, the policy mixes with a distribution u_r uniform over the current top- r experts and zero elsewhere:

$$\tilde{p} = (1 - \beta)p + \beta u_r, \quad \beta = \frac{d - m}{1/r - m} \quad (d > m). \quad (3.11)$$

If $d = m$, the original logits are returned unchanged. Otherwise the hook returns $\log \tilde{p}$ in the native logit dtype, after which the model performs its usual dispatch and weighting.

In exact arithmetic these mixtures reach the desired gap and preserve expert ordering away from singular endpoints. The implementation caps mixture coefficients below one to avoid fully uniform ties or zero probabilities. Finite precision can still introduce ties and small target errors, so the recorded diagnostics use the achieved margin after casting. Relative expert weights can change even when top- k membership does not. Learning a range from trace averages and applying it to individual token margins is a specific experimental assumption tested by the generation comparison.

3.4.4 Matched generation and evaluation

Each policy is evaluated against a freshly generated native-routing baseline on identical held-out prompts and attempt identifiers. The two conditions share

the checkpoint, tokenised inputs, chat-template options, sampling settings, per-attempt seeds, and completion budget. The default stochastic protocol uses temperature 0.6, top- $p = 0.95$, and 32,768 output tokens; repeated benchmarks retain their original attempt counts. Matching seeds pairs the randomness settings but does not force identical sampled trajectories once the distributions change.

Interventions apply during both prompt prefill and decoding at the policy’s selected layers. The generation engine uses eager execution, disables prefix caching and speculative decoding, and installs router hooks after warmup. Native expert dispatch remains responsible for executing the chosen experts. The run verifies that every intended hook processes tokens. Diagnostics record changed top- k sets and, for margin guiding, achieved margins and distance from the target range. Hook counters can include engine padding or recomputation and are not interpreted as independent generated tokens.

The comparison reports final-answer accuracy, generated-token counts, truncations, and paired wrong-to-right and right-to-wrong transitions. For N matched attempts, the empirical accuracy change is

$$\hat{\Delta}_\pi = \frac{1}{N} \sum_{i=1}^N (Y_i^\pi - Y_i^0) = \frac{N_{\text{wrong} \rightarrow \text{right}} - N_{\text{right} \rightarrow \text{wrong}}}{N}. \quad (3.12)$$

Token counts include reasoning and final-answer text. Benchmark-specific results accompany pooled comparisons, since pooling attempts gives more weight to repeated-attempt benchmarks. Matching manifests are checked before comparison so that changes in prompts, scoring, software, or sampling are not silently attributed to the routing policy.

Together, these experiments distinguish a calibrated association from the measured effect of acting on it. The method specifies the complete path from reference labels to held-out intervention evaluation; whether either policy improves accuracy or reduces token use is an empirical result, not an assumption of the pipeline.

Chapter 4

Experimental Setup

This chapter describes the computing environment, model implementations, datasets, and evaluation protocols used in the experimental work. The three principal studies address complementary questions: whether an automatic annotator can identify reasoning episodes, whether upcoming episodes can be predicted from hidden representations, and whether routing measurements are associated with answer correctness and reasoning events. Model weights remain fixed throughout these studies. The guiding stage additionally fits routing policies on calibration problems and evaluates them on held-out problems, as specified in Section 3.4.

4.1 Computing infrastructure

Most experiments were executed on the University of Bologna NLP group’s Slurm-managed GPU cluster, principally on `moro43`, the node equipped with one NVIDIA GeForce RTX 5090 with 32 GB of VRAM. Annotation experiments also used one RTX 3090 on `faretra`, the cluster’s master node. The cluster guide lists four RTX 3090 GPUs on that machine, each with 24 GB of VRAM; these are separate devices, rather than a single shared memory pool [29].

Table 4.1 describes node capacity, rather than a resource allocation guaranteed to each job. The GEPA and probe batch launchers request one GPU, 64 GiB of host memory, and a 48-hour time limit; the probe launcher also

Table 4.1: Resources of the principal experimental node. GPU specifications follow the cluster guide; CPU and host-memory capacities are the scheduler values displayed by the dashboard [29, 30].

Resource	Configuration
Compute node	moro43
GPU	One NVIDIA GeForce RTX 5090
GPU memory	32 GB
Scheduler CPU capacity	32 CPU resources
Scheduler host memory	Approximately 121 GB, as displayed
Execution environment	Linux, Slurm, and GPU-enabled Docker containers

explicitly requests four CPUs. Their retained default node is **faretra**, so reproducing those jobs on the main RTX 5090 machine requires overriding the node at submission. The sentence-tagging launcher requests one GPU, eight CPUs, 64 GiB, and four days. These limits are scheduler requests, not measurements of elapsed runtime or peak memory.

The cluster has independent node filesystems. Consequently, the repository, data, and Docker images must be available on the allocated node. The launchers mount the project into the container and use `/llms` as the shared model cache on that node, with `HF_HOME` pointing to the mounted directory [29]. Slurm’s assigned GPU identifiers are passed to Docker. Generation, optional annotation, and activation extraction execute sequentially so that their model weights do not occupy GPU memory together.

The 32 GB device-memory budget influenced the setup directly. Quantised weights made the chosen models practical to load, while long contexts still required space for inference caches and intermediate tensors. The extraction stages therefore retain boundary vectors or compact statistics instead of serialising every full activation tensor.

Table 4.2: Software configuration declared by the Dockerfile, project metadata, and current launchers. Version intervals are dependency constraints, not exact installed versions for every experiment.

Component	Declared configuration
Project container	Ubuntu 22.04, CUDA 12.8.1, Python 3.11
PyTorch	CUDA 12.8 wheels; no explicit version pin for <code>torch</code>
Transformers	$\geq 5.5.0$, < 6
GPTQ loading	GPTQModel 7.3.1
Replay backend	Unsloth 2026.8.18; bitsandbytes $\geq 0.46.1$
Prompt optimisation	DSPy $\geq 3.0.0$, < 4 ; GEPA $\geq 0.0.27$
Probe fitting	scikit-learn $\geq 1.5.0$
Mathematical scoring	<code>math-verify</code> $\geq 0.9.0$, < 1
vLLM server	<code>vllm/vllm-openai:v0.29.0</code>
GGUF annotation server	Locally built <code>llama.cpp:localcuda</code> image

4.2 Software and model configurations

The project runs in Python 3.11 inside a CUDA-enabled Docker image based on Ubuntu 22.04 with CUDA 12.8.1. PyTorch is installed from the CUDA 12.8 wheel index. Hugging Face Transformers provides model loading and forward execution, while NumPy, pandas, SciPy, and scikit-learn support data processing, statistical analysis, and probe training. DSPy and GEPA implement prompt optimisation. Table 4.2 records the relevant declared dependencies and image settings.

4.2.1 The subject model and the annotation models

The representation and routing studies use Qwen3.5-35B-A3B. Its language backbone has approximately 35 billion total parameters and 3 billion active parameters, 40 layers, and a hidden dimension of 2,048. Each MoE block has 256 routed experts, of which eight are selected per token, together with a shared expert. The backbone alternates Gated DeltaNet and gated-attention blocks [7]. The experiments use textual problems and continuations; they do

not evaluate visual inputs.

The annotator is a separate model. The Qwen3.6 GEPA experiments use Qwen3.6-27B in a GGUF Q4_K_XL configuration served through `llama.cpp`. The model-alignment study uses Qwen3.8-27B in UD-Q4_K_XL format. These models supply episode predictions and optimisation feedback; the routing measurements described in this chapter come from Qwen3.5-35B-A3B.

4.2.2 Quantisation and generation–replay provenance

The official Qwen/Qwen3.5-35B-A3B-GPTQ-Int4 checkpoint supplies native GPTQ 4-bit weights for the probe experiment. The correlation generation stage serves this same checkpoint identifier through vLLM. Correlation replay instead loads `unsloth/Qwen3.5-35B-A3B` through Unsloth with runtime 4-bit quantisation. Thus, the generator and replay model have corresponding base model identities, but different quantisation formats and execution engines.

Replay means supplying the saved problem and continuation to a frozen model in a teacher-forced forward pass. It does not generate a replacement answer. The resulting routing and hidden-state features describe how the replay implementation processes those tokens. They are not direct recordings of the generator’s internal activations, and numerical equivalence between the two implementations is not assumed.

4.3 Datasets and units of observation

4.3.1 Annotated reasoning traces

The annotation and probe studies use the released Schoenfeld reasoning corpus of Li et al. [9]. The repository snapshot contains 38 DeepSeek-R1 responses to SAT problems and 3,125 annotated units. Each unit has one of the seven labels introduced in Section 1.3. The SAT problem, answer choices, and available textual descriptions of tables or figures provide context. Reference answers and rationales are excluded from the annotator’s model-visible input.

The annotation study retains all 3,125 units. The probe study removes the

Table 4.3: Episode support in the local corpus. Only the synthetic closing units are removed for the probe experiment.

Episode	Annotation study	Probe study
Read	377	377
Analyze	791	791
Plan	225	225
Implement	675	675
Explore	208	208
Verify	629	629
Monitor	220	182
Total	3,125	3,087

38 synthetic `</think>` closing units, one per response, all labelled Monitor. Its retained corpus therefore contains 3,087 units. This removes a structural marker that could otherwise provide a shortcut for Monitor classification and matches the sentence count in the source study. Table 4.3 shows the resulting class distribution.

The two studies deliberately use different split protocols. GEPA partitions complete responses before flattening their units. With seed 42, the final-fit partition contains 26 training, six validation, and six test responses, corresponding to 2,382, 407, and 336 units. The probe replication instead uses a target-stratified 80/20 sentence split with seed 42: 2,469 training units and 618 test units for each binary target. All 38 response identifiers occur in both probe partitions. Consequently, the probe split measures within-corpus predictability and does not provide a response-disjoint generalisation estimate.

4.3.2 Mathematical reasoning benchmarks

The completed correlation run covers the six mathematical benchmarks in Table 4.4. The dataset selection and repeated-attempt convention follow the project’s adaptation of the SPIRAL evaluation suite [15]. There are 1,547 source problems and 4,647 generated attempts. These are corpus sizes before

Table 4.4: Benchmark coverage of the correlation corpus. Counts refer to source problems and generated attempts, not to the number of independent observations in every statistical analysis.

Dataset	Problems	Attempts per problem	Total attempts
MATH-500 [31]	500	1	500
AIME 2024 [32]	30	32	960
AIME 2025 [33]	30	32	960
OlympiadBench [34]	675	1	675
AMC 2023 [35]	40	32	1,280
Minerva Math [36]	272	1	272
Total	1,547	—	4,647

any analysis-specific filtering.

MATH-500 is loaded from HuggingFaceH4/MATH-500. AIME, AMC, and Minerva use the `math-ai` releases `aime24`, `aime25`, `amc23`, and `minervamath`. These loaders use the published `test` split. OlympiadBench uses Hothan/OlympiadBench, configuration `OE_T0_maths_en_COMP`, exposed by the repository loader as `train`. That split name belongs to the dataset release; these problems are used for evaluation without training the subject model. GPQA-Diamond and MMLU-Pro are supported by the pipeline but are absent from this completed six-benchmark corpus.

A separate qualitative probe-transfer stage uses 20 examples each from GSM8K, MATH, PRM800K, and ProcessBench. It teacher-forces their supplied solutions or reconstructed rated paths, rather than generating new ones. These 80 examples serve to inspect predicted episode labels and are not included in the correlation benchmark totals.

4.4 Episode-annotation protocol

Each annotation request contains the problem and the previous, current, and next response units. The model returns one episode label for the cur-

rent unit. The initial context-aware instruction contains 21 audited synthetic demonstrations, three per class, chosen to illustrate ambiguous category boundaries without copying held-out annotations. Since the next unit is visible, this protocol is an offline annotation procedure.

GEPA searches over instructions while keeping the model fixed [10]. The experiments include exact-match and inverse-frequency weighted rewards, followed by an informative same-model judge reward. In the latter configuration, the judge scores gold agreement, functional fit, contextual coherence, and boundary precision from one to five. Their weights are 0.40, 0.25, 0.15, and 0.20, respectively, and the weighted score is normalised for optimisation. Gold labels are available to this feedback stage, but are not included in the classifier request.

Prompt selection remains based on reference-label performance. The default selection measure is balanced accuracy; accuracy and macro-F1 are also evaluated. A candidate is retained only if it improves the selected measure and does not lower any class recall by more than 0.10 on validation data. This gate prevents a gain in frequent classes from concealing a substantial loss in a less frequent episode.

Configuration comparisons use five-fold nested response-grouped cross-validation over the 32 development responses, with the six test responses excluded. Each outer fold has its own inner training and validation partition. The final-fit studies then use the 26/6/6 partition described above. The Qwen3.8 alignment starts from the 40-example prompt selected by the Qwen3.6 fit, uses the heavy GEPA budget and medium reasoning effort, and retains the imported seed after the validation gate rejects the new candidate. The selected instruction is subsequently frozen for downstream annotation.

Reported classification measures include exact accuracy, balanced accuracy, macro-F1, per-class precision and recall, Cohen’s κ , and valid-output coverage. Kendall’s τ_b is retained in saved artifacts but is not a primary measure for the nominal episode categories. The final-fit runs do not evaluate the 336-unit test partition. Moreover, those responses had already been inspected in preliminary experiments, so their designation as a locked test does not make them a newly untouched evaluation set.

4.5 Prospective hidden-state probes

Probe extraction supplies each SAT problem as a user message and its released DeepSeek-R1 response as an assistant continuation. Annotated units are matched verbatim and monotonically to the response before tokenisation. For a unit beginning at token b_j , the stored feature is the hidden state at $b_j - 1$. The target unit and later text are therefore unavailable at the measurement boundary.

All 41 hidden-state indices are retained: index 0 is the embedding output and indices 1–40 are successive block outputs. Each feature has 2,048 dimensions. Following the replication protocol of Sun et al. [17], a separate one-versus-rest logistic regression is fitted for each label at each index, giving $7 \times 41 = 287$ classifiers. Training uses raw activations, balanced class weights, L2 regularisation with $C = 1$, the `lbfgs` solver, and at most 2,000 iterations. No standardisation or principal-component analysis is applied.

Accuracy, binary F1, and AUROC are computed on the 618-unit test partition. For each target, the selected index maximises accuracy on that same partition. This selection and the shared response identities make the best-index scores exploratory. In the transfer inspection, the seven selected binary classifiers are applied independently. Their scores do not constitute calibrated multiclass probabilities; choosing their maximum produces an inspection label rather than a reference annotation.

Only boundary activations are persisted. At float32 precision, the $3,087 \times 41 \times 2,048$ values require approximately 0.97 GiB, before metadata and classifier files. This storage choice retains the information required for refitting probes without preserving every token’s full hidden-state trajectory.

4.6 Benchmark generation and routing extraction

4.6.1 Generation settings

Benchmark solutions are generated zero-shot at temperature 0.6 and top- $p = 0.95$, with a completion budget of 32,768 tokens. The current launcher allocates a 49,152-token context, leaving room for the input prompt in addition to the completion. The default sampling seed is zero, with the per-request seed formed as $s_{i,a} = s_0 + 1000i + a$, where i is the problem index within a dataset and a is the attempt index. The saved trace records the actual seed and sampling settings.

The vLLM configuration permits eight concurrent requests and eight sequences, enables prefix caching, and uses a batch-token budget of 8,192. Generation uses the checkpoint’s MTP head with three speculative tokens. These are execution settings: request concurrency does not alter the declared number of attempts per problem, and long contexts can reduce the number of requests that decode simultaneously. The GPU memory utilisation setting is 0.90 and tensor parallelism is one for the main single-GPU configuration.

4.6.2 Replay measurements and layer selection

After generation, the inference server is stopped and the saved continuations are replayed through the corresponding Hugging Face model. Extraction retains decoder indices 27, 34, 35, 36, 38, and 39. This is the compatible union of indices selected by the separate probe experiment. Probe index 40 is excluded because decoder router indices run from 0 to 39.

The indexing convention is significant. A probe hidden-state index denotes a block output, while replay pairs the block-input hidden state at the retained decoder index with that block’s router distribution. The hidden vector is therefore not the exact normalised router input after the intervening sequence-mixing computation. Probe-based layer selection is a way to restrict measurement locations, not evidence that these locations are optimal intervention targets.

At each retained layer, extraction computes the routing and representation summaries defined in Chapter 1: entropy-derived concentration, top-1/top-2 margin, selected top-eight probability mass, selection-boundary margin, selected-versus-unselected mean probability gap, top-1 switching, top-eight set overlap, hidden-state norm, successive-state distance, and hidden/router geometry. Geometry calculations use at most 128 uniformly sampled tokens per trace by default. Full hidden and router tensors are reduced during extraction; compact scalar features and selected expert IDs are persisted. Original layer indices remain attached to the measurements.

The correlation analysis includes whole-continuation, full-reasoning, class-specific, and absolute-position views. Position analysis uses ten disjoint windows defined relative to the corpus mean reasoning length, followed by an overflow window. These are common absolute token windows, rather than ten equal fractions of each response. A shorter trace has no observation in a window it never reaches. Coverage is consequently reported alongside each view. Class-specific views use available sentence annotations. Full-reasoning and position views use all reasoning tokens independently of annotation coverage.

4.6.3 Sentence annotation

The pipeline applies the frozen GEPA annotator to generated reasoning. Its vLLM judge checkpoint is `unsloth/Qwen3.8-27B-NVFP4`, with temperature zero, low reasoning effort, a 4,096-token completion budget, and a 32,768-token context. The server uses eager execution and an FP8 KV cache; judge MTP is disabled in the default configuration. This deployment differs from the GGUF model and medium-effort setting of the alignment study, so agreement measured there is not automatically an estimate of downstream annotation accuracy.

Before tagging, the Qwen sampling protocol selects attempt zero for repeated problems and at most 100,000 sentences across benchmarks. Benchmark quotas are proportional to their observed error rates; sentences are sampled uniformly within each benchmark with seed 42. The sampling manifest identifies the selected units. This outcome-dependent sample has a different population from the full repeated-attempt corpus. Class-specific analyses produced from it

therefore describe the labelled subset rather than the full corpus.

4.7 Evaluation and statistical protocol

Mathematical answers are checked with `math-verify`; the pipeline records when a normalised exact or numeric fallback is used. For an attempt with a known reference answer, an unparseable response is counted as incorrect in the all-attempt evaluation. This denominator is distinct from an accuracy calculated only over successfully parsed answers.

For benchmarks with one attempt per problem, the reported score is empirical pass@1. For the repeated AIME and AMC evaluations, it is avg@32: the mean correctness over 32 attempts for each problem, averaged over problems. It does not measure whether at least one of the 32 attempts succeeds. A pooled attempt-level score gives more weight to benchmarks with more attempts, so benchmark-specific scores remain necessary for interpretation.

Continuous features are related to binary correctness or event indicators using point-biserial correlation. At the problem level, mean features are related to avg@32 using Spearman’s rank correlation. The primary repeated- problem analysis requires the full set of unique attempt identifiers, complete binary outcomes, and no duplicate generations under the recorded fingerprint checks. Problems with both correct and incorrect attempts also support a paired correct-minus-incorrect feature contrast.

The saved analysis uses 500 bootstrap resamples and seed 42 for 95% percentile intervals. Resampling occurs at the source-problem level, keeping repeated attempts together. Benjamini–Hochberg adjustment is applied within the declared testing families [13]; layerwise, feature-pair, and expert-use screens remain exploratory. The initial expert-use screen pools equal numeric expert indices across retained layers. It therefore cannot identify a particular layer-specific expert module without a subsequent analysis preserving the full (ℓ, e) identity.

Token-budget handling. The analysis retains supplied token-limit completions and records their termination status in a generation-budget audit.

Missing or nonfinite features are omitted only for the relevant analysis pair. Problem-level avg@32 requires a complete set of attempts and binary outcomes. Estimates remain conditional on the configured generation budget.

4.8 Artifacts, reproducibility, and experimental scope

The implementation is organised under `src/moe_exp/gepaLLMsJudge`, `src/moe_exp/probeTest`, and `src/moe_exp/correlation_pipeline`. Their outputs preserve the information needed to connect a numerical result to its inputs: split response identifiers, seed and selected prompts, per-call judge feedback, activation manifests, classifier parameters, generation settings, text fingerprints, retained layer indices, extraction summaries, and analysis-specific inclusion counts. Generation, annotation, and replay are checkpointed independently, allowing completed work to be reused when the recorded configuration matches.

Probe artifacts use the root `results/probeTest` and the run name `qwen3.5-35b-a3b-gptq-int4`. GEPA runs have separate directories under `results/gepaLLMsJudge`. Correlation outputs use the root `results/correlation_pipeline`, with the Qwen corpus under `reasoning-vllm-v1`. Saved fingerprints identify the generation, annotation, and replay inputs used by each analysis.

The manifests and launchers provide substantial provenance, but several dependencies still use version intervals, local image tags, or the mutable model revision `main`. These identify a configuration without fixing every underlying binary. Exact reproduction therefore also depends on the resolved model revision, Docker image digest, package versions, and matching code revision. Seeds alone do not establish bitwise equivalence across different quantisation or inference implementations.

The guiding stage is implemented in `src/moe_exp/moe_identity_guiding` and `src/moe_exp/moe_margin_guiding`. These workflows fit separate model-specific policies from the correlation artifacts and compare native and guided

generation on matched held-out problems. Their split, calibration, intervention, and evaluation rules are described in Section 3.4. Split manifests, policy files, per-condition generations, and routing diagnostics provide the provenance for a guiding result; implementation checks alone do not establish an improvement in generation accuracy.

Chapter 5

Results

Chapter 6

Conclusions

Bibliography

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022. URL: <https://proceedings.neurips.cc/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html>.
- [2] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/ed3fea9033a80fea1376299fa7863f4a-Abstract-Conference.html.
- [3] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations*. 2017. URL: <https://arxiv.org/abs/1701.06538>.
- [4] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: *Journal of Machine Learning Research* 23.120 (2022), pp. 1–39. URL: <https://jmlr.org/papers/v23/21-0998.html>.
- [5] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume

- Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gervet, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. *Mixtral of Experts*. 2024. DOI: 10.48550/arXiv.2401.04088. arXiv: 2401.04088 [cs.LG].
- [6] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models*. 2024. DOI: 10.48550/arXiv.2401.06066. arXiv: 2401.06066 [cs.CL].
- [7] Qwen Team. *Qwen3.5-35B-A3B Model Card*. 2026. URL: <https://huggingface.co/Qwen/Qwen3.5-35B-A3B> (visited on 09/07/2026).
- [8] Alan H. Schoenfeld. *Mathematical Problem Solving*. Academic Press, 1985. ISBN: 978-0-12-628870-4. DOI: 10.1016/C2013-0-05012-8.
- [9] Ming Li, Nan Zhang, Chenrui Fan, Hong Jiao, Yanbin Fu, Sydney Peters, Qingshu Xu, Robert Lissitz, and Tianyi Zhou. “Understanding the Thinking Process of Reasoning Models: A Perspective from Schoenfeld’s Episode Theory”. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2025, pp. 18267–18288. DOI: 10.18653/v1/2025.emnlp-main.922. URL: <https://aclanthology.org/2025.emnlp-main.922/>.
- [10] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. “GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning”. In: *International Conference on Learning Representations*. 2026. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/0e9e708b6f48e14fd0ac29e167413f76-Abstract-Conference.html.

- [11] John Hewitt and Percy Liang. “Designing and Interpreting Probes with Control Tasks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 2733–2743. DOI: 10.18653/v1/D19-1275. URL: <https://aclanthology.org/D19-1275/>.
- [12] Emmy Liu, Chenxuan Cui, Kenneth Zheng, and Graham Neubig. “Testing the Ability of Language Models to Interpret Figurative Language”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2022, pp. 4437–4452. DOI: 10.18653/v1/2022.naacl-main.330. URL: <https://aclanthology.org/2022.naacl-main.330/>.
- [13] Yoav Benjamini and Yosef Hochberg. “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing”. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 57.1 (1995), pp. 289–300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.
- [14] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. *Let’s Verify Step by Step*. 2023. DOI: 10.48550/arXiv.2305.20050. arXiv: 2305.20050 [cs.LG].
- [15] Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, Wee Sun Lee, and Natasha Jaques. *SPIRAL: Self-Play on Zero-Sum Games Incentivizes Reasoning via Multi-Agent Multi-Turn Reinforcement Learning*. 2025. DOI: 10.48550/arXiv.2506.24119. arXiv: 2506.24119.
- [16] John Hewitt and Christopher D. Manning. “A Structural Probe for Finding Syntax in Word Representations”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4129–4138.

- DOI: 10.18653/v1/N19-1419. URL: <https://aclanthology.org/N19-1419/>.
- [17] Lihao Sun, Hang Dong, Bo Qiao, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. “LLM Reasoning as Trajectories: Step-Specific Representation Geometry and Correctness Signals”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 26872–26887. DOI: 10.18653/v1/2026.acl-long.1237. URL: <https://aclanthology.org/2026.acl-long.1237/>.
- [18] Kang Chen, Sihan Zhao, Yixin Cao, and Yugang Jiang. *Beyond the Trace: Coupling an Interpretable Reasoning-State Readout to Native MoE Routing*. Preprint. 2026. DOI: 10.48550/arXiv.2608.17638. arXiv: 2608.17638.
- [19] Matthew Lyle Olson, Neale Ratzlaff, Musashi Hinck, Man Luo, Sungduk Yu, Chendi Xue, and Vasudev Lal. “Probing Semantic Routing in Large Mixture-of-Expert Models”. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 18263–18278. DOI: 10.18653/v1/2025.findings-emnlp.991. URL: <https://aclanthology.org/2025.findings-emnlp.991/>.
- [20] Charles Ye, Bo Yuan, and Lee Sharkey. *Polysemantic Experts, Monosemantic Paths: Routing as Control in MoEs*. Preprint. 2026. DOI: 10.48550/arXiv.2604.17837. arXiv: 2604.17837.
- [21] Mengru Wang, Xingyu Chen, Yue Wang, Zhiwei He, Jiahao Xu, Tian Liang, Qiuzhi Liu, Yunzhi Yao, Wenxuan Wang, Ruotian Ma, Haitao Mi, Ningyu Zhang, Zhaopeng Tu, Xiaolong Li, and Dong Yu. *Two Experts Are All You Need for Steering Thinking: Reinforcing Cognitive Effort in MoE Reasoning Models Without Additional Training*. 2025. DOI: 10.48550/arXiv.2505.14681. arXiv: 2505.14681 [cs.AI].
- [22] Mohsen Fayyaz, Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Ryan Rossi, Trung Bui, Hinrich Schütze, and Nanyun Peng. *Steering*

- MoE LLMs via Expert (De)Activation*. Version 2, February 2026. 2025. DOI: 10.48550/arXiv.2509.09660. arXiv: 2509.09660 [cs.CL].
- [23] Youngsik Yoon, Siwei Wang, Wei Chen, and Jungseul Ok. *When Are Experts Misrouted? Counterfactual Routing Analysis in Mixture-of-Experts Language Models*. 2026. DOI: 10.48550/arXiv.2605.07260. arXiv: 2605.07260.
- [24] Guinan Su, Yanwu Yang, Li Shen, Lu Yin, Shiwei Liu, and Jonas Geiping. *Rewiring Experts on the Fly: Continuous Rerouting for Better Online Adaptation in Mixture-of-Expert Models*. Preprint. 2025. DOI: 10.48550/arXiv.2510.14853. arXiv: 2510.14853.
- [25] Nina Rimskey, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. “Steering Llama 2 via Contrastive Activation Addition”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 15504–15522. DOI: 10.18653/v1/2024.acl-long.828. URL: <https://aclanthology.org/2024.acl-long.828/>.
- [26] Yihuai Hong, Meng Cao, Dian Zhou, Lei Yu, and Zhijing Jin. “The Reasoning-Memorization Interplay in Language Models Is Mediated by a Single Direction”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 21565–21585. DOI: 10.18653/v1/2025.findings-acl.1111. URL: <https://aclanthology.org/2025.findings-acl.1111/>.
- [27] Ningyu Xu, Qi Zhang, Chao Du, Qiang Luo, Xipeng Qiu, Xuanjing Huang, and Menghan Zhang. “Revealing Emergent Human-Like Conceptual Representations from Language Prediction”. In: *Proceedings of the National Academy of Sciences* 122.44 (2025), e2512514122. DOI: 10.1073/pnas.2512514122.
- [28] Tianhang Zhang, Lin Qiu, Qipeng Guo, Cheng Deng, Yue Zhang, Zheng Zhang, Chenghu Zhou, Xinbing Wang, and Luoyi Fu. “Enhancing Uncertainty-Based Hallucination Detection with Stronger Focus”. In:

- Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 915–932. DOI: 10.18653/v1/2023.emnlp-main.58. URL: <https://aclanthology.org/2023.emnlp-main.58/>.
- [29] University of Bologna NLP Group. *UniBoNLP GPU Cluster: Guide*. URL: <http://137.204.107.40:37339/slurm/guide> (visited on 09/13/2026).
- [30] University of Bologna NLP Group. *UniBoNLP Cluster Resource Allocation Dashboard*. URL: <http://137.204.107.40:37339/slurm/> (visited on 09/13/2026).
- [31] Hugging Face H4. *MATH-500: Dataset Release*. URL: <https://huggingface.co/datasets/HuggingFaceH4/MATH-500> (visited on 09/13/2026).
- [32] math-ai. *AIME 2024: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/aime24> (visited on 09/13/2026).
- [33] math-ai. *AIME 2025: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/aime25> (visited on 09/13/2026).
- [34] Hothan. *OlympiadBench: Dataset Release*. URL: <https://huggingface.co/datasets/Hothan/OlympiadBench> (visited on 09/13/2026).
- [35] math-ai. *AMC 2023: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/amc23> (visited on 09/13/2026).
- [36] math-ai. *Minerva Math: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/minervamath> (visited on 09/13/2026).